

Mesh Cross-Section Visualization: Implementation Report

Priyank

GV 271 – Graphics and Visualization, Indian Institute of Science • September 7, 2026

The assignment renders an OFF-format mesh, rotates it continuously about a randomly chosen axis, and reveals its internal structure by cutting it into $2M$ angular wedges pulled apart radially. Wedge decomposition is implemented twice against the same geometric definition: once rebuilt on the CPU every frame, and once inside a GPU geometry shader that clips the static mesh in place. A final section reports the CPU-vs-GPU performance comparison.

1 Task 1: Model Loading, Camera Setup, Rotation, and Base Rendering

(i) **Approach.** The OFF parser reads vertices/polygon indices into a flat indexed triangle buffer. Since OFF carries no normals, per-vertex normals are derived by accumulating *unnormalized* face cross products at each vertex and normalizing the sum – an area-weighted average for free, since a cross product’s magnitude is twice the triangle area. The camera auto-fits the model: the bounding-box diagonal gives a conservative bounding-sphere radius, from which camera distance and near/far planes follow. Projection is orthographic, with frustum half-extents corrected against window aspect so the model is never stretched. The mesh rotates continuously about an axis drawn uniformly at random once per run (`initRandomAxis`), advancing every frame at a fixed rate driven by wall-clock time (`timer`), composed into the model matrix with the centering translation and normalizing scale.

(ii) **Core files/functions.** `main.cpp`: `readOffFile`, `createVertexBuffers`, `createVertexNormals`, `ComputeModelTransform`, `setupCamera`, `updateProjection`, `initRandomAxis`, `timer`. `shader.vs/shader.fs` (`shaderProgram`) render the whole mesh.

(iii) **Challenges / design decisions.** Deriving normals from the mesh itself made the area-weighting free – summing unnormalized cross products gives this at no extra cost. Undistorted orthographic view across resizes required branching on `aspect` ≥ 1 and expanding the longer axis. A random rotation axis can, by chance, land close to the camera’s fixed view direction, under which rotation produces almost no visible depth change and the object appears to spin flat rather than tumble in 3D; this was fixed by rejecting and redrawing any candidate axis within roughly 32° of the view direction.

2 Task 2: Depth-Based Colormap Shading

(i) **Approach.** Lighting uses Blinn-Phong (ambient + diffuse + half-vector specular). The diffuse base color comes from a depth-driven colormap (viridis or coolwarm) instead of a flat material, so once the mesh is cut open in Tasks 3–4, interior faces are distinguished by depth without needing a second material.

(ii) **Core files/functions.** `shader.fs`: `colormapViridis`, `colormapCoolWarm`, lighting in `main()`. `shader.vs`: `vDepth01` from `-viewPos.z`. Uniforms `uKa/uKd/uKs/uShininess`, `uNearDepth/uFarDepth` set once in `main.cpp`.

(iii) **Challenges / design decisions.** Lighting was evaluated per-fragment in shaders rather than on the CPU specifically for performance: every quantity it depends on (rotated normal, view vector, depth color) changes every frame under continuous rotation regardless of implementation, and the GPU evaluates this in parallel across thousands of cores, avoiding both a serial per-pixel CPU loop and re-uploading a full color buffer every frame. Depth uses view-space `-viewPos.z` (not NDC z) so the ramp stays linear regardless of projection type. Colormaps are piecewise-linear keyframe tables, trading exact color fidelity for shader simplicity. A `uUseVertexColorLoc` toggle isolates shading bugs from geometry bugs during debugging.

3 Task 3: CPU-Side Wedge Slicing

(i) **Approach.** M evenly spaced lines through the origin ($\theta_i = i\pi/M$) give $2M$ rays around the full circle; each wedge is bounded by two half-planes from its boundary rays rotated $\pm 90^\circ$. Every triangle is clipped against both half-planes in sequence (a Sutherland–Hodgman clip specialized to triangles), and surviving geometry per wedge is pushed outward along its bisector for the exploded view; all buffers rebuild every frame. At $M = 1$ the two wedges are further distinguished by polygon mode – one `GL_FILL`, one `GL_LINE` – satisfying the base solid/wireframe requirement; at $M > 1$ all wedges are filled.

(ii) **Core files/functions.** `main.cpp`: `rebuildWedgeGeometryDefs` (angles, half-plane normals, bisectors), `clipListAgainstPlane` (lone-vertex detection, edge-intersection, 1-or-2-triangle output), `lerpVert`, `buildAllWedgesCPU`, `uploadSliceVAO`.

(iii) **Challenges / design decisions.** The “lone” vertex is found without sorting: with three booleans and two possible values, at least two must agree, so a pigeonhole case split locates the odd one out in $O(1)$. The intersection parameter $t = d_a/(d_a - d_b)$ is guaranteed in $(0, 1)$ once the lone vertex’s sign is confirmed opposite its neighbors. Indices are cyclically relabeled ($a, b, c = \text{lone}, (\text{lone}+1)\%3, (\text{lone}+2)\%3$), preserving winding so lighting/culling stay correct post-clip. The cost is performance: the full mesh is re-clipped and every wedge’s VBO/EBO re-uploaded every frame, motivating Task 4.

4 Task 4: GPU-Side Wedge Slicing via Geometry Shader

(i) **Approach.** The same clip algorithm runs as a per-triangle geometry-shader stage in local space, reusing one static buffer every frame with no CPU rebuild. A pass-through vertex shader forwards local position/normal; the geometry shader clips against `uWedgeNormalA`, then `uWedgeNormalB` (two chained half-space clips = the wedge), applying MVP and the offset only to emitted vertices. The Task 3 fill/wireframe rule at $M = 1$ is mirrored here via `glPolygonMode` around the GPU draws.

(ii) **Core files/functions.** `slice.vs` (pass-through). `slice.gs`: `clipTri`, `main()` (two-stage clip chaining), `emitWorld/emitTri`. `shader.fs` reused unchanged. `main.cpp`: `buildPreppedWholeMesh`, uniform wiring for `sliceGSProgram`.

(iii) **Challenges / design decisions.** `max_vertices=12` covers the worst case: up to 2 triangles from the first clip, each yielding up to 2 more, times 3 vertices. Normals are re-normalized after interpolation, since interpolating two unit vectors isn't itself unit length. The key bug was variable shadowing: locally re-declaring `uWedgeNormalA/BLoc` during init silently left the intended global locations unassigned, so clip-plane uniforms stayed at GLSL's default (0, 0, 0), discarding every triangle. Relatedly, uniform storage is per-program, not per-name – lighting uniforms set only on `shaderProgram` left the GPU-sliced mesh pure black until set again on `sliceGSProgram`.

5 Performance Analysis: CPU vs. GPU Wedge Slicing

(i) **Approach.** FPS is measured by accumulating frames over fixed wall-clock windows (`glutGet(GLUT_ELAPSED_TIME)` in `timer`), shown in the window title. Two axes are varied: active planes M (1–10, via `j/k`, calling `rebuildWedgeGeometryDefs`) and mesh resolution (separate OFF files). Each (resolution, M) pair is logged for both CPU and GPU modes, with non-parallel plane orientations.

(ii) **Core files/functions.** `main.cpp`: `timer`, `keyboard (j/k)`, `display (mode dispatch)`.

(iii) **Expected behavior, reasoning, and results.** CPU cost scales with (triangle count $\times 2M$ wedges), since `buildAllWedgesCPU` re-clips and re-uploads every wedge every frame. GPU cost grows far more gently with M : each wedge costs one more draw call plus inherently parallel `slice.gs` work, with no re-clip or bus transfer – `buildPreppedWholeMesh`'s buffer uploads once, regardless of M . Higher resolution scales the CPU loop directly and serially, while the GPU's per-primitive invocation is embarrassingly parallel and degrades more slowly until cores saturate. Wedge boundaries crossing the mesh's dense interior force more triangles through the expensive split-and-interpolate branch on both implementations – cheap on the GPU (a few extra `EmitVertex` calls in GPU memory), costlier on the CPU (more interpolation and a bigger re-upload).

Mesh	Vertices	Triangles	Planes (M)	CPU FPS	GPU FPS
1GRM_1	83673	167346	1	5.1	282.9
1GRM_1	83673	167346	2	4.4	246.1
1GRM_1	83673	167346	4	3.2	152.2
1GRM_1	83673	167346	6	2.7	106.7
1GRM_1	83673	167346	8	2.4	78.6
1GRM_1	83673	167346	10	2.0	67.6
1GRM_2	41836	83672	1	9.9	473.6
1GRM_2	41836	83672	2	8.7	445.1
1GRM_2	41836	83672	4	6.5	248.2
1GRM_2	41836	83672	6	5.5	196.9
1GRM_2	41836	83672	8	4.6	148.4
1GRM_2	41836	83672	10	3.8	117.7
1GRM_4	20918	41836	1	18.5	745.7
1GRM_4	20918	41836	2	16.5	685.7
1GRM_4	20918	41836	4	11.8	401.7
1GRM_4	20918	41836	6	10.1	325.7
1GRM_4	20918	41836	8	8.8	270.3
1GRM_4	20918	41836	10	8.0	220.8
1GRM_8	10459	20918	1	35.4	988.6
1GRM_8	10459	20918	2	31.9	968.6
1GRM_8	10459	20918	4	23.5	717.1
1GRM_8	10459	20918	6	20.1	495.0
1GRM_8	10459	20918	8	17.1	427.4
1GRM_8	10459	20918	10	15.2	371.8
1GRM_16	5229	10458	1	70.7	990.0
1GRM_16	5229	10458	2	59.2	991.4
1GRM_16	5229	10458	4	47.0	957.1
1GRM_16	5229	10458	6	39.4	791.4
1GRM_16	5229	10458	8	33.1	597.1
1GRM_16	5229	10458	10	27.0	540.7
20AR_1	519221	1038654	1	0.7	73.2
20AR_1	519221	1038654	2	0.7	58.3
20AR_1	519221	1038654	4	0.5	30.3
20AR_1	519221	1038654	6	0.4	21.2
20AR_1	519221	1038654	8	0.3	16.4
20AR_1	519221	1038654	10	0.3	13.2
20AR_2	259557	519326	1	1.5	107.8
20AR_2	259557	519326	2	1.2	109.4
20AR_2	259557	519326	4	1.0	59.2
20AR_2	259557	519326	6	0.8	41.4
20AR_2	259557	519326	8	0.7	31.3
20AR_2	259557	519326	10	0.6	25.7
20AR_4	129725	259662	1	2.8	196.0
20AR_4	129725	259662	2	2.5	198.6
20AR_4	129725	259662	4	2.0	111.0
20AR_4	129725	259662	6	1.8	77.8
20AR_4	129725	259662	8	1.4	61.3
20AR_4	129725	259662	10	1.3	48.5
20AR_8	64810	129830	1	5.6	340.0
20AR_8	64810	129830	2	5.3	346.6
20AR_8	64810	129830	4	4.1	201.7
20AR_8	64810	129830	6	3.1	136.2
20AR_8	64810	129830	8	2.7	109.2
20AR_8	64810	129830	10	2.5	91.2
20AR_16	32361	64914	1	12.0	606.3
20AR_16	32361	64914	2	9.4	570.0
20AR_16	32361	64914	4	7.2	327.1
20AR_16	32361	64914	6	6.6	244.7
20AR_16	32361	64914	8	5.5	190.1
20AR_16	32361	64914	10	4.4	162.9
3SY7_1	221469	442946	1	1.6	141.8
3SY7_1	221469	442946	2	1.5	124.3
3SY7_1	221469	442946	4	1.1	66.9
3SY7_1	221469	442946	6	1.0	47.9
3SY7_1	221469	442946	8	0.9	36.4
3SY7_1	221469	442946	10	0.8	29.9
3SY7_2	110733	221472	1	3.1	219.1
3SY7_2	110733	221472	2	2.8	214.0
3SY7_2	110733	221472	4	2.2	124.8
3SY7_2	110733	221472	6	1.9	88.4
3SY7_2	110733	221472	8	1.7	67.0
3SY7_2	110733	221472	10	1.4	55.3
3SY7_4	55365	110736	1	6.7	385.7
3SY7_4	55365	110736	2	6.0	372.3
3SY7_4	55365	110736	4	4.7	216.5
3SY7_4	55365	110736	6	3.7	156.5
3SY7_4	55365	110736	8	3.4	121.6
3SY7_4	55365	110736	10	2.7	102.7
3SY7_8	27680	55368	1	13.4	645.7
3SY7_8	27680	55368	2	12.0	608.6
3SY7_8	27680	55368	4	8.5	380.9
3SY7_8	27680	55368	6	7.1	278.8
3SY7_8	27680	55368	8	6.8	214.3
3SY7_8	27680	55368	10	5.7	176.6

Summary

The CPU implementation is easier to debug – state is inspectable directly on the host – at the cost of re-deriving and re-uploading geometry every frame. The GPU implementation removes that cost by clipping in place on a static buffer, at the cost of a fixed output-vertex budget and uniform-management bugs (shadowed locations, per-program state) with no CPU-side analogue. This same per-program uniform-state lesson recurred in shading (Task 2/4) and rotation-axis sampling (Task 1): most substantive bugs here were in state management connecting CPU setup to GPU execution, not in the core geometric algorithms.